5. Neural Networks, Part 1

Neurons, layers, and training

Week 5 · CE/ARCE 2XX AI Literacy for Civil and Architectural Engineering



COLLEGE OF ENGINEERING
Civil & Architectural Engineering & Mechanics

Agenda

Today's plan

1. Quiz on last week (Week 4)
2. From a line to a neuron
3. Layers and the forward pass
4. How networks learn
5. Case study: heating load of buildings
6. Lab 05 preview and wrap-up

Recap

Last week in 60 seconds

1 Walk downhill

Gradient descent repeats $w \leftarrow w - \alpha \cdot \partial J/\partial w$ (and the same for b) until the cost J stops falling.

2 Learning rate α

Too small: painfully slow. Too big: the steps overshoot and the cost blows up.

3 Test on unseen data

Train on one part of the data; report MAE, RMSE, and R^2 on a test set the model never saw.

4 Overfitting

A model that fits the training data too closely does worse on new data. Simple models can underfit.

Learning objectives

By the end of today, you can…

- ✓ Explain a neuron as a weighted sum plus an activation, and why the activation must be nonlinear
- ✓ Compute a forward pass by hand for a tiny network and count its parameters
- ✓ Describe training in words: loss, gradient descent, backpropagation, epochs, and batches
- ✓ Recognize overfitting in a learning curve and explain early stopping
- ✓ Compare a neural network with linear regression on building heating load

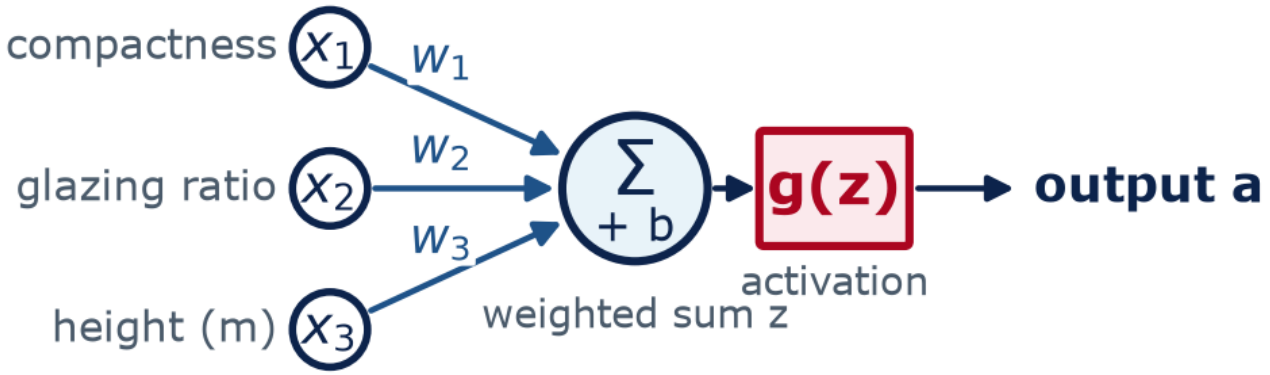
01

From a line to a neuron

Week 3's straight line, plus one bend

From a line to a neuron

A neuron is Week 3's line plus one extra step



Same as Week 3

Weighted sum of the inputs plus a bias: $z = w_1x_1 + w_2x_2 + w_3x_3 + b$

New: an activation

g(z) bends the result before it is passed on

Many neurons

Connected in layers, neurons form a **neural network**

From a line to a neuron

One neuron, in two short formulas

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b \qquad a = g(z)$$

z	weighted sum — the same thing as Week 3's $\hat{y} = w \cdot x + b$
$w_1 \ldots w_n$	weights: how strongly each input pushes the output
b	bias: shifts the result up or down
g	activation function: adds a bend (next slides)
a	the neuron's output, passed on to the next layer

Without a bend, stacked layers collapse into one straight line

Two linear layers, no bend:

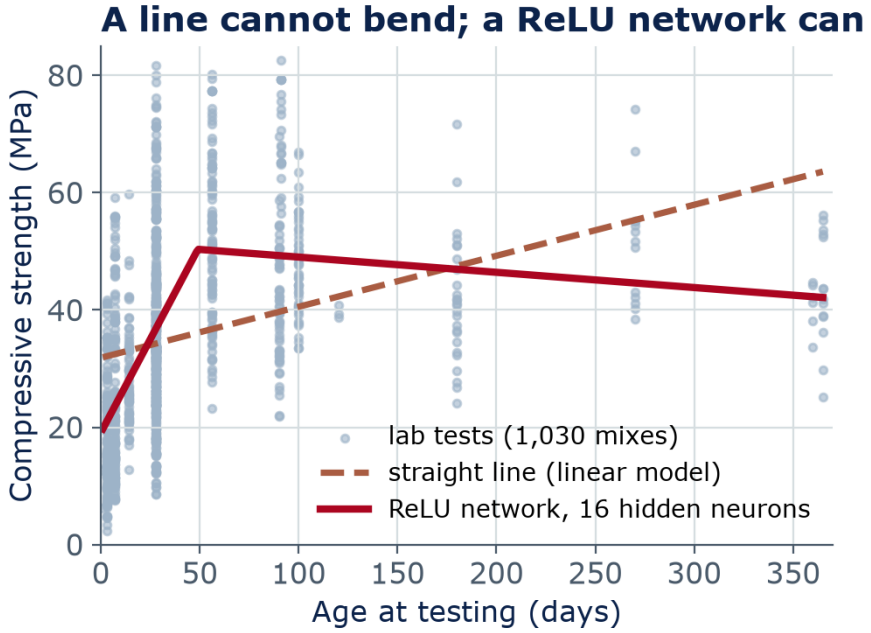
Layer 1: $h = 2x + 1$

Layer 2: $\hat{y} = -0.5h + 3$

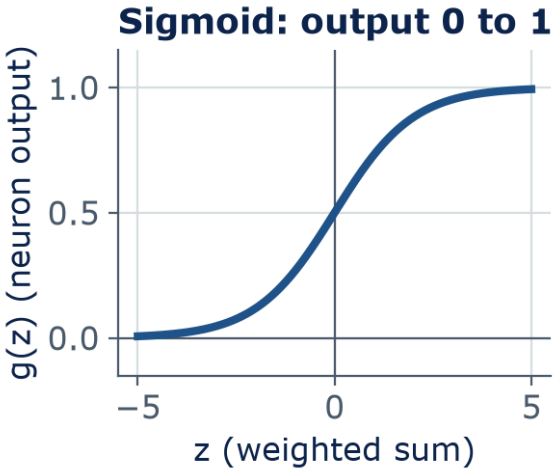
Combine: $\hat{y} = -0.5(2x + 1) + 3$

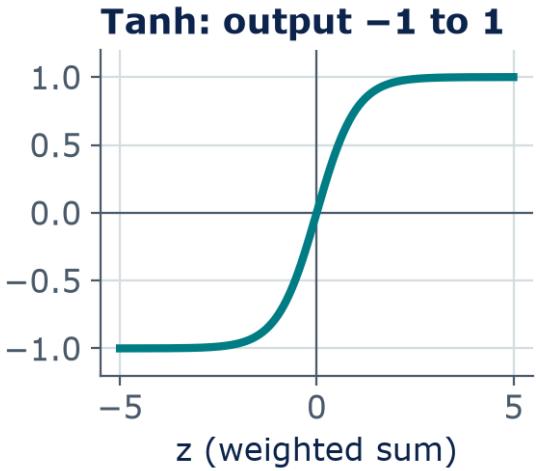
$= \mathbf{-x + 2.5} \rightarrow$ still a line

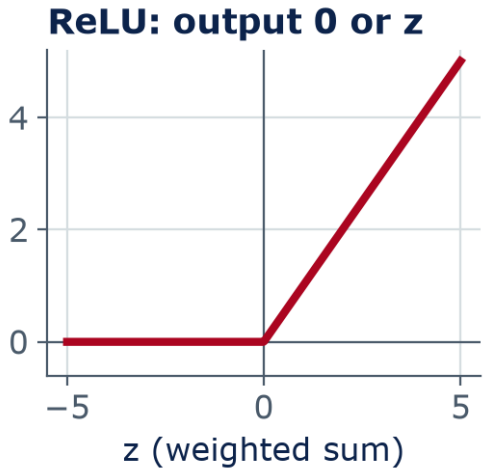
Put a nonlinear activation (like ReLU) between layers, and the network can **bend** to fit curves.



Activation functions add the bend — ReLU is today's default







Sigmoid

Squashes to 0–1. Used for probabilities (Week 4's logistic regression).

Tanh

Squashes to −1 to 1, centered at zero.

ReLU

max(0, z): negative → 0, positive passes. Simple and fast.

Concept check

What can a stack of linear layers represent?

A network has three layers, but NO activation functions — each neuron just computes w·x + b. What can it represent?

A Any curve, if the layers are wide enough

B Only a straight-line (linear) relationship

C Only on/off step functions

D Nothing — it cannot be trained

Concept check

What can a stack of linear layers represent? — answer

A network has three layers, but NO activation functions — each neuron just computes w·x + b. What can it represent?

A Any curve, if the layers are wide enough

B Only a straight-line (linear) relationship

C Only on/off step functions

D Nothing — it cannot be trained

A line of a line is still a line: the layers collapse into one linear model. The activation is what lets a network bend.

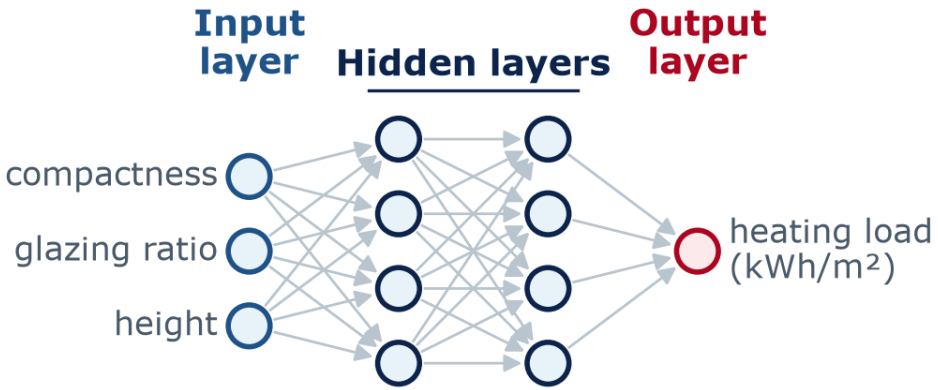
02

Layers and the forward pass

Multiply, add, bend — layer by layer

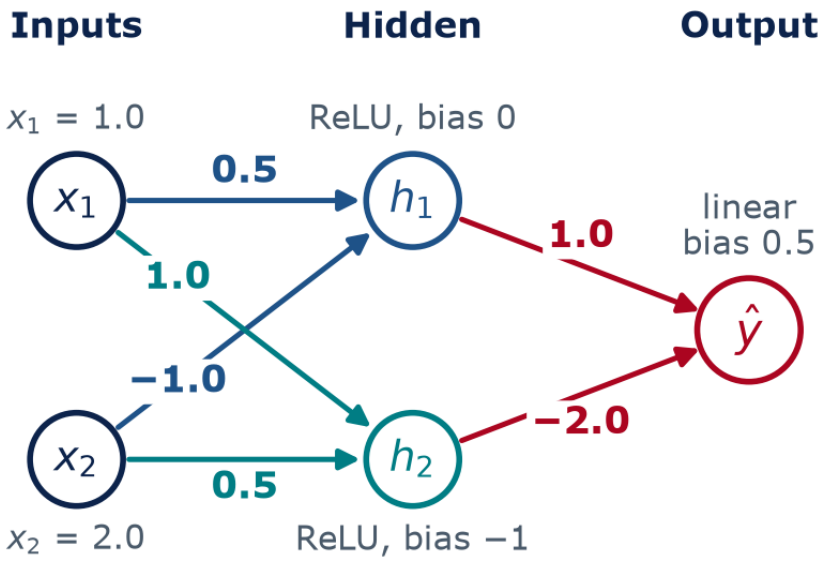
Neurons are organized in layers

- **Input layer:** the features — nothing is computed here
- **Hidden layers:** build new combinations of the inputs
- **Output layer:** the prediction (kWh/m²)
- Each neuron connects to all neurons in the layer before: a **multilayer perceptron (MLP)**
- **Deep** learning = many hidden layers



A 3–4–4–1 network for heating load

Your turn: run a tiny network by hand



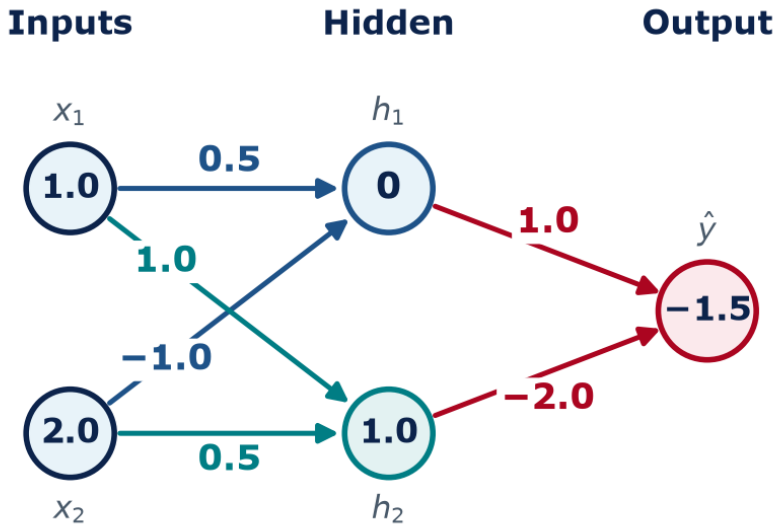
3 min

Work in pairs. Compute, in order:

1. $h_1 = \text{ReLU}(0.5 \cdot x_1 - 1.0 \cdot x_2 + 0)$
2. $h_2 = \text{ReLU}(1.0 \cdot x_1 + 0.5 \cdot x_2 - 1)$
3. $\hat{y} = 1.0 \cdot h_1 - 2.0 \cdot h_2 + 0.5$

ReLU(z) = z if z > 0, otherwise 0. The output neuron is linear: no ReLU.

The forward pass: multiply, add, bend — one layer at a time



Hidden neuron 1

$z_1 = 0.5 \cdot 1.0 - 1.0 \cdot 2.0 + 0 = -1.5$

$h_1 = \text{ReLU}(-1.5) = \mathbf{0}$

Hidden neuron 2

$z_2 = 1.0 \cdot 1.0 + 0.5 \cdot 2.0 - 1 = 1.0$

$h_2 = \text{ReLU}(1.0) = \mathbf{1.0}$

Output (linear)

$\hat{y} = 1.0 \cdot 0 - 2.0 \cdot 1.0 + 0.5$

$\hat{y} = \mathbf{-1.5}$

$h_1 = 0$ means neuron 1 is "off" for this input.

Counting parameters: weights + biases, layer by layer

Each layer: (inputs × neurons) weights + one bias per neuron

Network or layer	Weights	Biases	Parameters
Our tiny network: hidden layer (2 neurons × 2 inputs)	2 × 2 = 4	2	6
Our tiny network: output layer (1 neuron × 2 inputs)	2 × 1 = 2	1	3
Our tiny 2–2–1 network, total	6	3	**9**
Linear regression with 8 building inputs	8	1	9
Heating-load network 8 → 32 → 16 → 1 (today's case study)	784	49	833
Two outputs (heating + cooling): 4 → 3 → 3 → 2 = 15 + 12 + 8	27	8	35
Small language model from Lab 01 (Qwen2.5-0.5B)			≈ 0.5 billion

Concept check

Count the parameters

A network has 3 inputs, one hidden layer with 4 neurons, and 1 output neuron. How many parameters (weights + biases) does it have?

A 12

B 16

C 20

D 21

Concept check

Count the parameters — answer

A network has 3 inputs, one hidden layer with 4 neurons, and 1 output neuron. How many parameters (weights + biases) does it have?

A 12

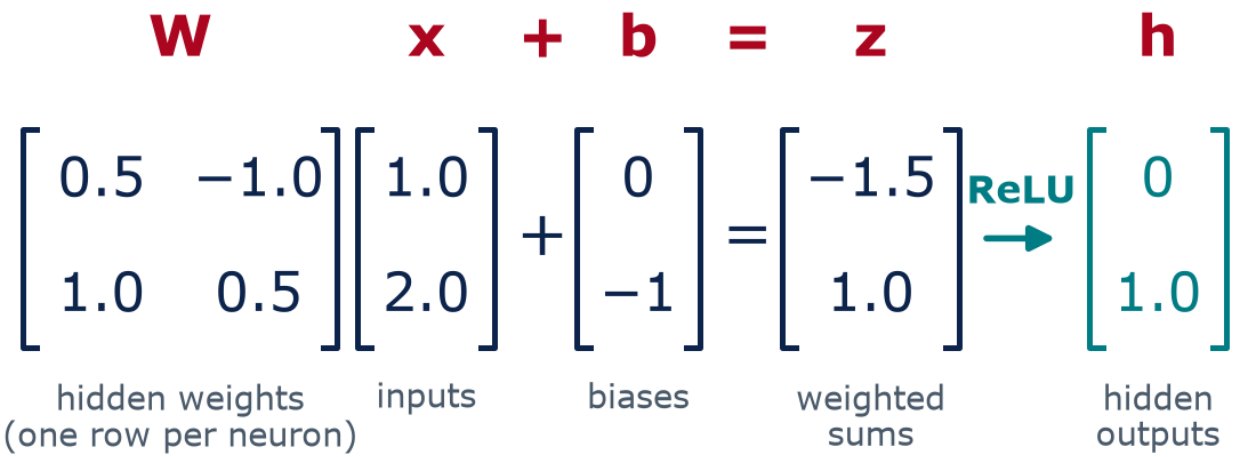
B 16

C 20

D 21

Hidden: 3 × 4 weights + 4 biases = 16. Output: 4 weights + 1 bias = 5. Total = 21.

The matrix view: a whole layer in one line



- A layer = **h = ReLU(W x + b)** — one row of W per neuron
- Graphics processors (GPUs) do millions of these multiply-adds at once

The same forward pass in NumPy (Lab 05)

```
import numpy as np
x  = np.array([1.0, 2.0])          # inputs
W1 = np.array([[0.5, -1.0],        # hidden neuron 1
               [1.0,  0.5]])       # hidden neuron 2
b1 = np.array([0.0, -1.0])         # hidden biases
w2 = np.array([1.0, -2.0])         # output weights
b2 = 0.5                           # output bias

z = W1 @ x + b1        # weighted sums -> [-1.5, 1.0]
h = np.maximum(0, z)   # ReLU          -> [ 0.0, 1.0]
y_hat = w2 @ h + b2    # output        -> -1.5
print(y_hat)
```

- `@` means matrix multiply: all weighted sums at once
- `np.maximum(0, z)` is ReLU for every neuron
- Same numbers as the hand calculation: $\hat{y} = -1.5$
- In Lab 05 you change a weight and predict what happens

03

How networks learn

Same recipe as Week 4: loss + gradient descent

How networks learn

Training a network uses the same recipe as Week 4

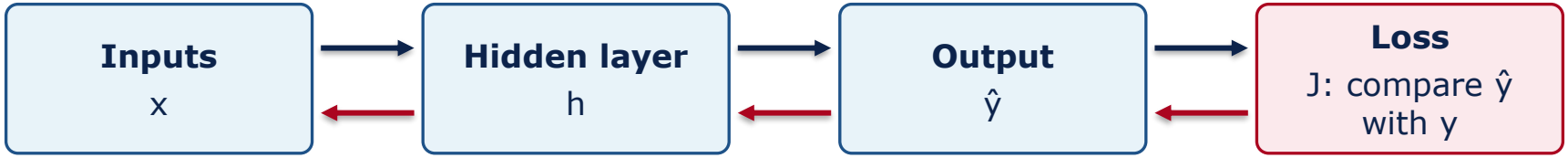


Predict	Loss	Blame	Update	Repeat
Forward pass: run the training examples through the network to get predictions	Measure the error, e.g., average squared error J (Week 3)	Backprop finds ∂J/∂w: how much is each weight to blame?	Step every weight downhill with Week 4's rule: w ← w − α·∂J/∂w	Many passes (epochs) until the validation error stops improving

Model + cost + gradient descent — the network just has more weights to adjust

Backpropagation passes the blame backward, layer by layer

→ Forward pass: inputs flow to a prediction and a loss



← Backward pass: the error flows back as "blame" (∂J/∂w) for each weight

Chain rule, in words

Blame on a hidden weight = (blame on ŷ) × (how much ŷ depends on h) × (how much h depends on that weight)

Engineering analogy

Over budget? Ask each subcontractor: if your part changed a little, how much would the overrun change? That sensitivity is the blame — it can say "do more", not only "do less".

How networks learn

You choose the knobs; training chooses the weights

MAX_ITER

Epoch

One full pass through all training examples. Training usually takes many epochs.

BATCH_SIZE

Batch

A small group of examples (e.g., 32 buildings) used for one update step.

LEARNING_RATE_INIT

Learning rate α

Step size, exactly as in Week 4: too big diverges, too small crawls.

HIDDEN_LAYER_SIZES

Width

Neurons per hidden layer. More = more flexible, slower, easier to overfit.

HIDDEN_LAYER_SIZES

Depth

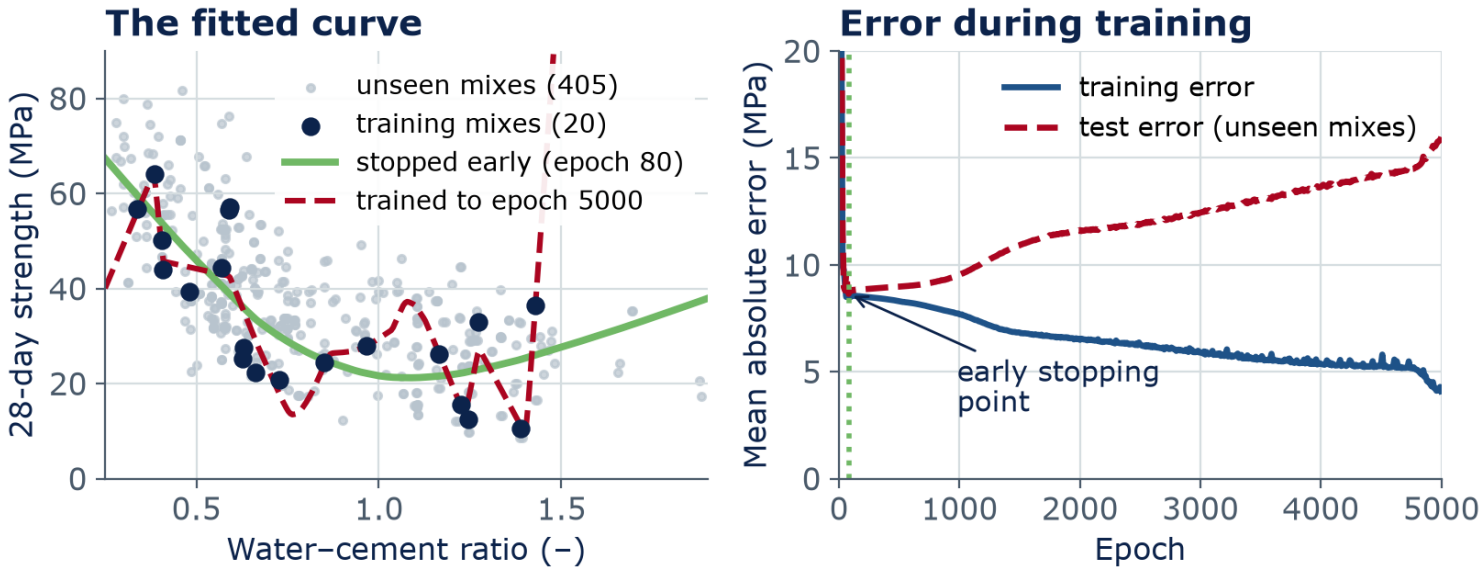
Number of hidden layers. (32, 16) means two hidden layers.

ACTIVATION

Activation

ReLU in hidden layers by default; a linear output for regression.

Overfitting: the network starts memorizing — so stop early



Fixes: **stop early** · use a smaller network · get more data · penalize large weights (regularization)

TensorFlow Playground: train a network in your browser

7 min

Demo · Pairs

- Open **playground.tensorflow.org** (laptop or phone)
- Circle data, **0 hidden layers**, press play. Can a straight boundary separate the dots?
- Add **1 hidden layer** with 4 ReLU neurons. What changes?
- Try the **spiral** with 2–3 hidden layers; watch the test loss
- Set training data to 10% and noise to 50: compare training and test loss

Report back: the smallest network that solves the circle, and what happened to the test loss with very little training data.

04

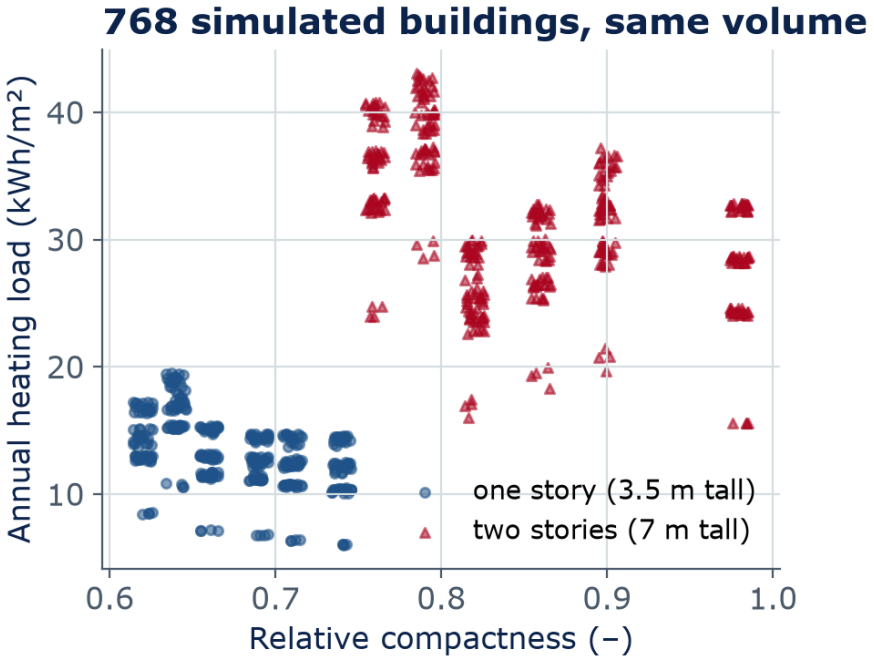
Case study: heating load

Predicting a building's heating demand from its shape

Case study: heating load

Can a network predict heating load from building geometry?

- **Data:** 768 building shapes simulated with energy software (Tsanas & Xifara, 2012)
- Same volume and materials; only shape, glazing, and orientation change
- **Inputs (8):** compactness, surface, wall and roof areas, height, orientation, glazing area and layout
- **Target:** annual heating load (kWh/m²)
- Loads form clusters and steps, not one straight trend



Case study: heating load

Training the network takes a few lines of scikit-learn

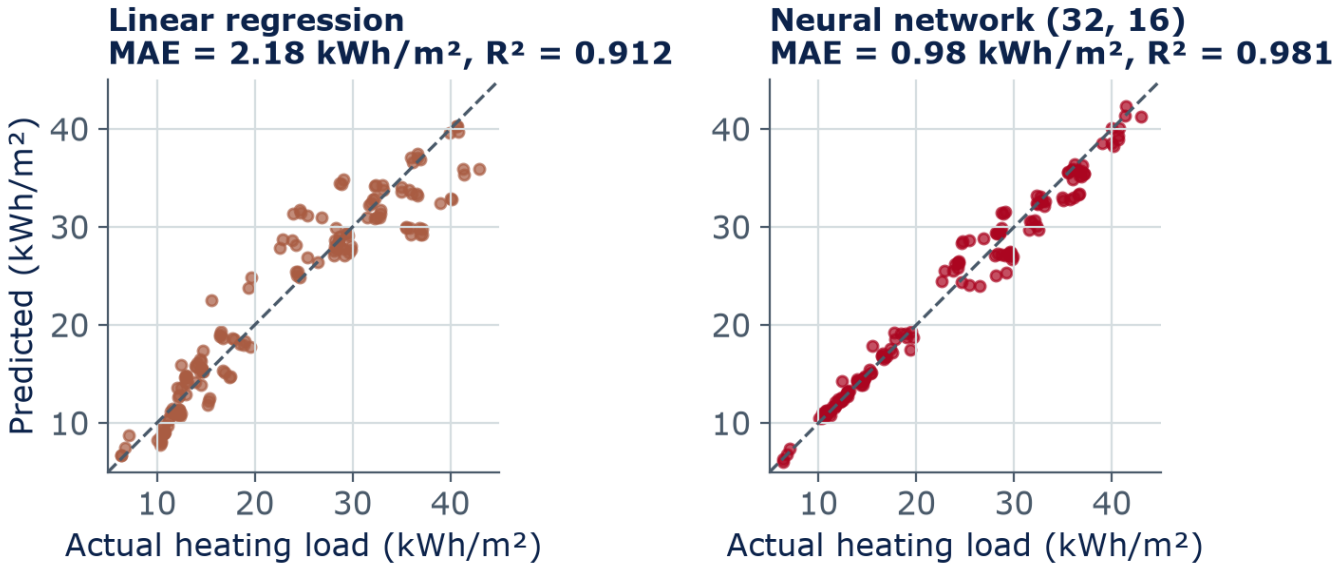
```
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

df = pd.read_csv(get_data("building_energy_efficiency.csv"))
y = df["heating_load_kwh_m2"]
X = df.drop(columns=["heating_load_kwh_m2",
                     "cooling_load_kwh_m2"])
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)
sc = StandardScaler().fit(X_tr)          # scale features
net = MLPRegressor(hidden_layer_sizes=(32, 16),
                   max_iter=3000, random_state=42)
net.fit(sc.transform(X_tr), y_tr)
```

- Same split and scaling as Week 4
- `(32, 16)`: two hidden layers, 32 and 16 neurons
- `max_iter`: the maximum number of epochs
- `fit` runs forward pass, backprop, and updates for you

Case study: heating load

On unseen buildings, the network cuts the error by more than half



Same data, same split. Linear regression: 9 parameters. Network: 833 parameters — more flexible, harder to explain.

Case study: heating load

Use a neural network only when a simple model is not enough

A network is a good fit

- Nonlinear relationships and interactions
- Hundreds or thousands of examples
- Many inputs, or raw data like images and signals
- Accuracy matters more than a readable formula

?

Think twice

- Small datasets (tens of points)
- A physics equation already works (heat loss = $U \cdot A \cdot \Delta T$)
- You must explain every coefficient to a reviewer
- Predictions far outside the training data

Always compare with a **baseline** (the simplest sensible model, e.g., linear regression) on test data

Lab 05

Lab day: your first neural network

LAB

05

Bring a charged laptop

Open the notebook from the course GitHub page

- TensorFlow Playground: guided experiments, answers recorded in the notebook
- Today's 2–2–1 forward pass in NumPy — check that you get $\hat{y} = -1.5$
- Train `MLPRegressor` on heating load and compare it with linear regression
- Vary layers and neurons; plot the loss curve; compare train and test error
- Exit ticket (3 AI-Free questions, last 10 min) · **HW2 due Sunday 11:59 pm**

Wrap-up

Key takeaways

1. A **neuron** = weighted sum + activation; a **network** = layers of neurons
2. Without a nonlinear activation, any stack of layers is still one straight line
3. **Forward pass:** multiply, add, bend, layer by layer. Parameters = weights + biases
4. **Training** = loss + gradient descent; **backpropagation** passes blame backward via the chain rule
5. Watch train vs. test error, **stop early**, and always compare with a simple baseline

Learn more

Readings and resources

- **3Blue1Brown — But what is a neural network?** — video, 19 min: neurons, layers, and weights, beautifully animated
- **3Blue1Brown — Backpropagation, intuitively** — video, 13 min: the 'passing blame backward' idea in pictures
- **Google ML Crash Course — Neural networks** — about 1 hour, with interactive exercises
- **TensorFlow Playground** — keep experimenting after class
- **scikit-learn user guide — Neural network models** — reference for MLPRegressor (Lab 05)